



VIGOLIUM

// What is Vigolium?

SERIOUS SECURITY AUDIT, NOT JUST A PR REVIEW

Catch critical vulnerabilities that traditional scanners and AI code review tools miss, with validated proof your team can act on.



www.vigolium.com / github.com/vigolium/vigolium

Whoami

- Known as **j3ssie**. Senior Security Engineer at TikTok
- **Open-source lover** building security tools for practitioners.
- **Recognized by Fortune 500 leaders** including Google, Apple, Microsoft, Netflix, Snapchat, the US Department of Defense, and many more for identifying security vulnerabilities in their products.

[illegible]

DAST is a crowded market

Odds are your AppSec stack already runs one of these. Tools you have all seen before we look at what is actually inside one.



Best Dynamic Application Security Testing Platforms



Commercial platforms

Invicti / Acunetix · Rapid7 InsightAppSec · Qualys WAS ·
Checkmarx DAST · HCL AppScan · Veracode DAST · AppCheck ·
Detectify · Intruder

Open source and practitioner tools

OWASP ZAP · Burp Suite (Pro / Enterprise) · Nuclei · Nikto ·
Wapiti · sqlmap

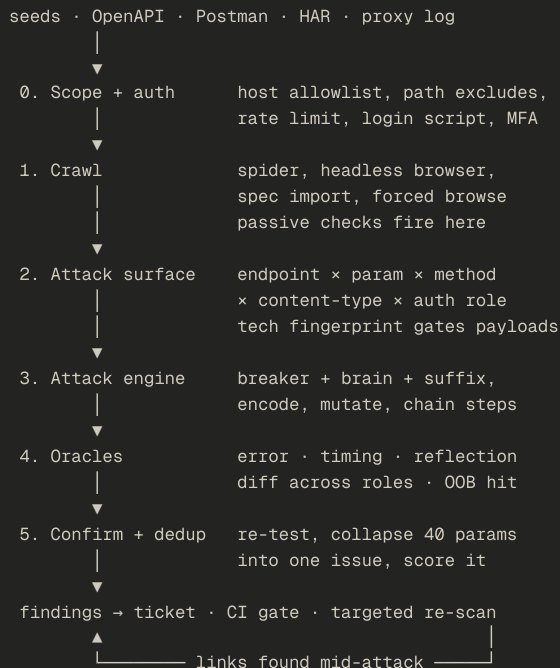
CI-native / API-first

StackHawk · Probely · Escape

Different vendors, one shared shape: crawl, fire a payload bank, score the response. **Next slide breaks that shape down**, because the differences live in coverage and confirmation, not in the vulnerability list.

What a DAST actually is

Black-box testing of a **running** app, with no source. It talks to the live HTTP, API and UI endpoints the way an attacker would.



What decides quality

Not the HTTP client. **Crawler**, **auth/session manager**, **attack-surface model**. A scanner that never sees `/admin/users/export` cannot find IDOR there.

How it knows a payload worked

Payload = **breaker** (`'`, `../`, `${`) + **brain** (`SLEEP(10)`, `alert(1)`, collaborator hit) + **suffix**. An oracle rules on it: error string, timing, reflection into a DOM sink, differential response across roles, or an OOB callback.

Proof, not guess. Regex on "syntax error" is a guess. Re-proving a 10s delay twice is proof.

DAST: pros and cons

Pros

- **Framework agnostic.** Same engine against Java, Go, Node, or a third-party UI
- **Tests what actually ships.** Proxies, cookies, CORS, debug endpoints, WAF gaps
- **Exploitability signal.** A confirmed SQLi delay or SSRF callback beats a dataflow warning
- **No source needed.** Works on legacy and third-party apps
- **Repeatable once scoped.** Nightly staging scan, PR smoke test, regression pack

Cons

- **Coverage is the crawl.** Unlinked routes, feature flags, CAPTCHA/MFA stay invisible
- **Blind to business logic.** "Transfer \$1 to myself twice" is not in the payload bank
- **Needs a stable env.** Staging drift causes both FPs and FNs, and prod is risky
- **URL + param, not** `file:line`. Devs still hunt for the sink
- **Environment-sensitive oracles.** Timing oracles lie on a loaded cluster

Where DAST sits in the stack

	SAST	DAST	IAST	RASP
View	Source	Outside, running	Inside + tests	Inside, production
Gives you <code>file:line</code>	✓	✗	✓	✓
Typical FP rate	High	Medium (low if proof-based)	Low	Very low
Coverage bound	Whole repo	What was crawled	What tests ran	Live traffic

They are complementary. But none of them knows what the app is *supposed to do*. That gap is where the real bugs live.

The gap

Continuous delivery never got continuous security testing

- **Manual pentests** ship as a PDF every 6 to 12 months. Stale before the next sprint, and nobody re-runs them on the diff
- **Traditional DAST/SAST** scan from a fixed checklist. No application context, no exploit synthesis, and false-positive rates that need a full-time triage owner
- **Diff-scoped AI reviewers** read the patch, not the running app. They emit prose, never an HTTP request
- **Vibe-coded apps** ship faster every month. Teams generate more production code than they review

Nothing in the standard stack reads the **whole repo**, **plans an attack**, **fires live HTTP probes**, and **validates the result** end to end. That's the slot Vigolium fills.

AI code review is not security testing

A diff reviewer reasons over a patch. A scanner attacks a running system.

Scope

AI code reviewers see the PR's lines plus a small retrieved context window.

Vigolium reads the whole repo – every route, auth flow and downstream service – then binds it to the live target.

A cross-endpoint auth bypass is not visible from one file.

Action

AI code reviewers reason about source and emit suggestions.

Vigolium synthesises payloads, fires live HTTP at the target, reads the response.

Static reasoning can't prove exploitability. A confirmed request/response pair can.

Signal

AI code reviewers produce unvalidated prose.

Vigolium produces a finding with request, response and a reproduction command.

Diff annotations are hints. HTTP records are evidence.

Diff-scoped AI review is a linter. Useful next to runtime exploitation, useless in place of it.

Architecture

Three components. The CLI does the work, the other two just read it.

Vigolium CLI

Single Go binary. Holds the **native scanner** (324 modules), the **olium runtime** and the **agent subcommands**. Also runs as a REST API server.

olium is the in-process LLM client. No sidecar, no SDK subprocess.

Vigolium Workbench

Self-hosted dashboard over the same DB the CLI writes to.

Static report

Portable HTML with request/response evidence, for offline review and sharing.



The goal: build systems that don't become obsolete.
Give agents the best native tools, and every new model release makes the system stronger.

Scanning modes

Two top-level commands. Both write to the same store and the same schema.

```
vigolium scan
```

Deterministic. Native Go pipeline, no LLM in the loop.

- 6 phases, 324 modules
- Three strategies: `lite` / `balanced` / `deep`
- Three intensities: `quick` / `balanced` / `deep`
- Repeatable, fast, CI-friendly

Best for: breadth across many applications.

```
vigolium agent <mode>
```

AI-driven. Dispatch through the in-process olium runtime.

- Source-aware via `--source` (git URL or local path)
- Bring your own provider or OAuth session

Best for: pre-release audits, logic-flaw hunting.

Boundary: native is the execution layer, agentic is the planning and triage layer. The agent picks modules and writes extensions, the native engine runs them.

Native scan: `vigolium scan`

Pipeline (6 phases)

1. Heuristics	WAF, redirects, tech fingerprint
2. External Harvest	Wayback, CommonCrawl, OTX, URLScan
3. Spidering	Chromium SPA crawler, form filling
4. Discovery	Brute-force dirs/files, JS analysis
5. Dynamic-Assess	207 active + 117 passive modules
6. KnownIssueScan	Nuclei templates + secret scanning

`lite` runs phase 5 only. `balanced` adds 3, 4 and 6. `deep` adds the external harvest.

`run probe` is the standalone host sweep: one request per target, passive fingerprint + surface score, no fuzzing.

Module dispatch

Each module declares its scope as a bitmask:

- **Host**. Once per host (fingerprint, default creds)
- **Request**. Once per request (headers, cookies)
- **InsertionPoint**. Once per parameter (SQLi, SSTI, XSS)

Engine internals

- **Value-aware mutation**. Classify each parameter by semantic type, then generate payloads for that type
- **OAST** callbacks catch blind SSRF, XXE and RCE
- Worker pool with per-host rate limiting

One binary replaces the recon stack

Vigolium takes these tools' **own argv** and routes it to the native phase. No wrapper scripts, no separate installs, and every result lands in the same store with the same finding and HTTP-record schema.

Instead of	Shim	Native
ffuf / feroxbuster	vigolium ffuf ...	run discovery --discovery-wordlist · fuzz https://t/FUZZ -w file-long
nuclei	vigolium nuclei ...	run known-issue-scan -t <url>
katana / gospider	vigolium katana ...	run spidering -t <url>
gau / waybackurls	vigolium gau <domain>	kit harvest <domain>
arjun	vigolium arjun -u <url>	fuzz --fuzz param-name --anomaly <url>
nikto	none	117 passive modules + run known-issue-scan
httpx	none	run probe -T hosts.txt (aliases httpx , alive , sweep) · --tls-probe
subfinder / amass	none	not covered. Pass a host list with -T

Better than chaining them: one scope model, one rate limiter, one session context, one dedup pass. The crawler's output feeds the attack surface directly instead of round-tripping through text files.

Sits inside your proxy: native Burp Suite and Caido plugins. Hotkey a request straight to ingestion, native scan, or agentic scan. Or run `vigolium server` and scan proxy traffic as it arrives.

A better DAST: any application shape

Classic DAST assumes a server-rendered app with an HTML login form. Almost nothing ships that way now. Vigolium **detects the shape first**, then picks the phases and payloads that can actually land.

Application shape	Where classic DAST stalls	What Vigolium does
SPA · React, Vue, Next, Nuxt	No links in the HTML source	Chromium spider as a state machine over DOM snapshots – CDP event-listener detection, automatic form filling
API-only , no UI at all	Nothing to crawl	Ingests OpenAPI, WSDL/SOAP, Postman, HAR, cURL, raw HTTP · one request per operation
Auth-gated , multi-step login	Session dies at the first redirect	Session engine: <code>login.steps</code> , token extract from JSON, cookie, header or regex, re-auth mid-scan
Multi-tenant / role-based	Can't tell a real 403 from someone else's 200	<code>primary</code> + <code>compare</code> sessions diff the same request across two identities
Behind a WAF or CDN	Blocked early, then blind	Proactive edge pacing, per-host limiter, reactive back-off · a host that starts filtering is reported, not hammered
Unknown stack	Fires every payload everywhere	Tech fingerprint gates the registry – framework, CMS and cloud packs only when they apply

Shape detection is a phase, not a guess. `--heuristics-check advanced` probes the root, reads the content type and detects the SPA framework, then drops phases that could never yield. A JSON-only API never pays for a browser crawl; an HTML monolith never wastes the budget on spec ingestion.

A better discovery tool: a loop, not a wordlist

Path discovery, JS request extraction and a browser crawler feed each other.

The loop

1. **Evidence before dictionary.** Observed filenames, paths and extensions outrank any wordlist; `/FUZZ` is the fallback
2. **Learned soft-404s.** Per-host signatures, plus a prefix circuit breaker so a catch-all route can't eat the hour
3. **Typed JS request facts.** Method, query fields, body template, content type, provenance – not an endpoint string
4. **Specs folded in.** API specs spotted in responses expand into request records
5. **Targeted re-spider.** Rich new pages get browsed after path discovery

vs. `katana` – where it actually differs

Katana headless renders JS, clicks bounded `onclick` handlers, captures POST bodies and filters near-duplicate URLs. "We drive a browser" is not the argument.

- **A state graph, not a link queue.** States keyed on normalised DOM, actions as edges, path replay. Three panels behind a menu at one unchanged URL is three states
- **Interaction context survives.** Export carries source and destination state, event type, selector, frame, submitted inputs
- **Forms it has to build.** Bounded GET/POST submission, autoscroll, iframe priming, follow-up passes

Stated honestly: architectural difference, not a measured one – no head-to-head crawl benchmark yet, and there is no single universal scope, limiter or dedup pass, just shared data contracts.

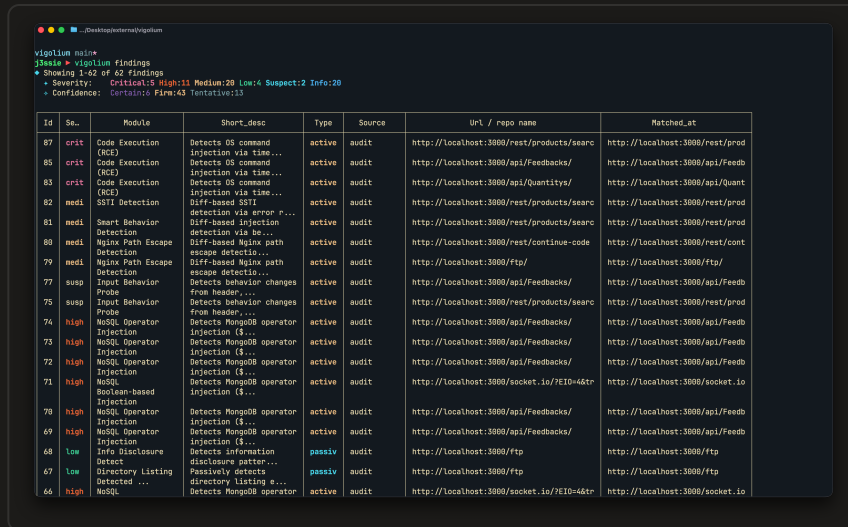
Inputs & integrations

What you point Vigolium at, how it authenticates, where it plugs in.

Input formats

- **URL / target list.** `-t https://example.com`, file, or stdin
- **OpenAPI / Swagger.** JSON or YAML · **WSDL / SOAP**
- **Postman.** Collection export
- **Burp Suite.** XML state file, or the Burp plugin
- **HAR.** Browser or proxy capture
- **cURL.** Raw `curl ...` command
- **Raw HTTP.** Request/response file · **Nuclei** JSONL

```
GET /catalog?searchTerm=apple HTTP/2
Host: localhost:3000
...
```



The screenshot shows the Vigolium terminal interface. At the top, it says "Vigolium main" and "Please * Vigolium Findings". Below this, it shows "Showing 1-62 of 62 findings" and a summary: "Severity: Critical:15 High:11 Medium:20 Low:4 Suspect:2 Info:20" and "Confidence: Certain:6 Firm:43 Tentative:15". Below the summary is a table with the following columns: Id, Se., Module, Short_desc, Type, Source, Url / repo name, and Matched_at. The table contains 62 rows of findings, each with a unique ID, severity level, module name, short description, type, source, URL, and matched timestamp.

Id	Se.	Module	Short_desc	Type	Source	Url / repo name	Matched_at
87	crit	Code Execution (RCE)	Detects OS command injection via time...	active	audit	http://localhost:3000/rest/products/search	http://localhost:3000/rest/prod
85	crit	Code Execution (RCE)	Detects OS command injection via time...	active	audit	http://localhost:3000/api/feedbacks/	http://localhost:3000/api/feeb
83	crit	Code Execution (RCE)	Detects OS command injection via time...	active	audit	http://localhost:3000/api/quantities/	http://localhost:3000/api/quant
82	medi	SSRF Detection	Diff-based SSRF detection via error f...	active	audit	http://localhost:3000/rest/products/search	http://localhost:3000/rest/prod
81	medi	Smart Behavior Detection	Diff-based injection detection via be...	active	audit	http://localhost:3000/rest/products/search	http://localhost:3000/rest/prod
80	medi	Nginx Path Escape Detection	Diff-based Nginx path escape detectio...	active	audit	http://localhost:3000/rest/continue-code	http://localhost:3000/rest/cont
79	medi	Nginx Path Escape Detection	Diff-based Nginx path escape detectio...	active	audit	http://localhost:3000/ftp/	http://localhost:3000/fts/
77	susp	Input Behavior Probe	Detects behavior changes from header...	active	audit	http://localhost:3000/api/feedbacks/	http://localhost:3000/api/feeb
75	susp	Input Behavior Probe	Detects behavior changes from header...	active	audit	http://localhost:3000/rest/products/search	http://localhost:3000/rest/prod
74	high	NoSQL Operator Injection (\$...)	Detects MongoDB operator injection (\$...	active	audit	http://localhost:3000/api/feedbacks/	http://localhost:3000/api/feeb
73	high	NoSQL Operator Injection (\$...)	Detects MongoDB operator injection (\$...	active	audit	http://localhost:3000/api/feedbacks/	http://localhost:3000/api/feeb
72	high	NoSQL Operator Injection (\$...)	Detects MongoDB operator injection (\$...	active	audit	http://localhost:3000/api/feedbacks/	http://localhost:3000/api/feeb
71	high	NoSQL Operator Injection (\$...)	Detects MongoDB operator injection (\$...	active	audit	http://localhost:3000/socket.io/?EIO=4&tr	http://localhost:3000/socket.io
70	high	NoSQL Operator Injection (\$...)	Detects MongoDB operator injection (\$...	active	audit	http://localhost:3000/api/feedbacks/	http://localhost:3000/api/feeb
69	high	NoSQL Operator Injection (\$...)	Detects MongoDB operator injection (\$...	active	audit	http://localhost:3000/api/feedbacks/	http://localhost:3000/api/feeb
68	low	Info Disclosure Detect	Detects information disclosure patter...	passiv	audit	http://localhost:3000/ftp	http://localhost:3000/fts
67	low	Directory Listing Detect	Passively detects directory listing e...	passiv	audit	http://localhost:3000/ftp	http://localhost:3000/fts
66	high	NoSQL Operator Injection (\$...)	Detects MongoDB operator injection (\$...	active	audit	http://localhost:3000/socket.io/?EIO=4&tr	http://localhost:3000/socket.io

Source code (for agentic modes)

`--source <local-path | git-url>` is the same flag across `swarm`, `autopilot`, `audit` and `query`. Vigolium clones git URLs into the session dir.

Sessions and multi-role testing

Sessions teach the scanner who it is logged in as. Two roles give you access-control testing.

```
# idor-test.yaml → vigolium scan --auth-file idor-test.yaml
sessions:
- name: admin
  role: primary          # drives crawl + discovery
  headers:
    Cookie: "${ADMIN_SESSION_COOKIE}"

- name: regular_user
  role: compare          # replayed against every
  headers:               # request the primary found
    Cookie: "${USER_SESSION_COOKIE}"

# Login flow with token extraction
- name: api_admin
  role: primary
  login:
    url: https://app/api/auth/login
    method: POST
    body: '{"u":"admin","p":"admin123"}'
    type: bearer
    token_path: .data.access_token
```

Why two roles matter

The `primary` session crawls. Every request it finds is replayed as each `compare` session and the responses diffed. An admin-only order returning 200 for a regular user is an IDOR, and no payload bank finds that.

Staying logged in

`login` blocks re-authenticate when a session expires mid-scan, pulling the new token from the JSON with `token_path`. Without it, half a long scan runs unauthenticated. `${ENV_VARS}` resolve at load, so the file is safe to commit.

JavaScript extensions

A scanner module you write in JS. Same shape as the built-in Go modules, hot-loaded, no rebuild.

```
// webhook_callback_detector.js → ~/.vigolium/extensions/
module.exports = {
  id: "webhook-callback-detector",
  type: "passive",          // or "active"
  severity: "suspect",
  confidence: "tentative",
  scope: "response",
  scanTypes: ["per_request"], // per_host | per_insertion_point

  scanPerRequest: function(ctx) {
    var ct = ctx.response.headers["content-type"] || "";
    if (ct.indexOf("json") === -1) return null;

    var hits = vigolium.utils.regexExtract(
      ctx.response.body,
      "(?:callback_url|webhook)\\s*:\\s*(https?://[^\"]+)"
    );
    if (!hits.length) return null;

    return [{ url: ctx.request.url, matched: hits.join(", "),
      name: "Webhook/Callback URL exposed",
      severity: "suspect" }];
  }
};
```

The APIs you get

`vigolium.http` shares the native rate limiter and session. `.db` reads stored traffic, `.parse` handles JSON, XML and JWTs, `.utils` regex and encoding, `.oast` callback payloads, `.agent` the LLM.

Same dispatch as the 324 Go modules, same `scanTypes` bitmask.

Where swarm comes in

The master agent writes these in its plan phase. If a target needs a check that doesn't exist yet, it writes one and the native engine runs it in the same scan.

Coverage: core vulnerability classes

OWASP Top 10 and beyond, broken out by the categories the scanner registry actually uses.

Injection

XSS (reflected, DOM-based, SSR hydration) · SQLi (error-based, boolean/time-blind) · NoSQLi (Mongo/Couch/Cassandra) · SSTI/CSTI · CRLF · OS command exec · XXE · prototype pollution · insecure deserialization

File & path

LFI · path traversal (null byte, double encode, Unicode) · file upload flaws · directory listing · backup discovery

Access control

CSRF · IDOR · authorization bypass · mass assignment · 403/401 forbidden bypass · HTTP method tampering

API & protocol

GraphQL introspection · SSRF (in-band + blind) · open redirect · request smuggling · JWT vulns · JSONP callback · WebSocket · race conditions

Coverage: stack-specific and out-of-band

Framework-specific

Spring Boot (actuators) · Django · Laravel · Rails ·
Express · Next.js · Nuxt · Remix · ASP.NET / Blazor ·
Flask · FastAPI

CMS

WordPress (XML-RPC, user enum, AJAX) · Drupal · Joomla ·
CMS installer exposure

Cloud & infra

Firebase (RTDB, storage, auth, functions) · cloud storage
listing/takeover · default credentials · web cache
poisoning · CORS misconfig · host header injection

Out-of-band (OAST)

Blind SSRF · blind XXE · blind RCE · OAST probes · proxy
pingback. A DNS or HTTP callback confirms each one, with
no in-band response needed.

Recent additions: MCP scanner suite (14 modules), `sensitive-header-leak`, `dom-xss-taint`, `xss-dom-confirm`, `surface-scoring`. Run `vigolium module ls mcp` or `vigolium module ls --type active` for the live registry.

Agentic modes: `vigolium agent <mode>`

7 subcommands. Scan / query / TUI dispatch through the in-process **olium** runtime; `audit` drives the **vigolium-audit** and **piolium** harnesses.

Subcommand	Family	Job
<code>autopilot</code>	agentic scan	One long LLM session with full Bash/Read/Write/Grep/Glob until the scan halts. Free-form pentest.
<code>swarm</code>	agentic scan	Multi-phase pipeline. Master agent picks modules and writes JS extensions; native engine executes; optional triage + rescan loop.
<code>audit</code>	source audit	Whitebox repo audit, no live target. Modes set phase count: <code>lite</code> (3) · <code>balanced</code> (9) · <code>deep</code> (12). <code>--driver auto audit piolium both</code> .
<code>triage</code>	triage	Confirm one finding with an AI triager; downgrades to info on a false positive.
<code>query</code>	one-shot	Single prompt → structured output. Code review, endpoint and secret discovery.
<code>olium</code>	TUI	Interactive multi-turn chat, provider-agnostic. Not orchestrated scanning. Alias <code>vigolium ol</code> .
<code>session</code>	sessions	List and inspect agent run sessions and their artifacts.

olium providers. OpenAI – `compatible` (default),
`api-key`, `responses`, `codex-oauth` · Anthropic –
`compatible`, `api-key`, `oauth`, `cli`, `vertex`

Session artifacts in `~/.vigolium/agent-sessions/` ·
`schemas` `findings`, `http_records`, `attack_plan`,
`triage_result`, `source_analysis`

Deep audit: 12 phases

`vigolium agent audit --mode deep --source <repo>` runs a full whitebox audit on the **vigolium-audit** harness (claude/codex). Multi-agent, bounded concurrency.

#	Phase	One-liner
D1	Dependency Intel	Advisory and dependency intelligence across the dependency tree.
D2	Commit Archaeology	Mine git history for silent security fixes and undisclosed CVEs.
D3	Patch Bypass	Re-derive identified security patches and test them for completeness.
D4	Threat Model	DFD/CFD slices, trust boundaries, knowledge base.
D5	Structural SAST	CodeQL + Semgrep, structural extraction, multi-service edge enumeration.
D6	Deep Probe	Systematic probe teams over every attacker-controlled input component.

#	Phase	One-liner
D7	Authorization Audit	Enumerate routes and handlers, build the authz matrix.
D8	Review Chambers	State/concurrency + spec review, adversarial debate, variant expansion, FP validation.
D9	Intent Reconciliation	Diff surviving findings against repository-documented intent.
D10	PoC & Partition	Consolidate drafts, author PoCs, split executed from theoretical.
D11	Cold Verification	Zero-context <code>report.md</code> re-derived fresh for every finalized finding.
D12	Report Assembly	<code>final-audit-report.md</code> assembled from both finding buckets.

Without local git history, D2 and patch-bypass are skipped explicitly; the source-snapshot phases carry on.

vs. commercial DAST

	Acunetix	Invicti	Rapid7 InsightAppSec	Tenable WAS	Vigolium
Crawl + active DAST	✓	✓	✓	✓	✓
Proof-based confirmation	✓	✓	⚠	⚠	✓
Whole-repo source context	✗	✗	✗	✗	✓
AI plans the attack	✗	✗	✗	✗	✓
User-authored modules	✗	✗	⚠	✗	✓ JS, hot-loaded
Drop-in for ffuf / nuclei / katana	✗	✗	✗	✗	✓
Burp / Caido live workflow	✗	✗	✗	✗	✓
Runs air-gapped, no seat/asset licence	✗	✗	✗	✗	✓

Where they still win: compliance reporting packs (PCI, HIPAA), enterprise SSO/RBAC at scale, and a support SLA. Vigolium competes on **depth per target**, meaning source context, planned attacks and replayable proof. Not on procurement checkboxes.

How we know a finding is real

Three filters sit between a payload landing and a finding reaching the store.

1. Runtime proof

A finding carries the HTTP request, the response, the module ID, the payload and a confidence rating. Replay it from the CLI or Workbench and get the same result.

Confidence is graded: **certain** means the payload executed, **firm** means behaviour changed, **tentative** means a heuristic matched.

2. Adversarial verification

In deep audits, one set of agents argues a finding is real and another argues it isn't. Anything that survives goes to a **cold verify** pass, where a fresh agent with no prior context re-checks it from scratch.

Disagreement kills the finding.

3. Disclosure

Findings from open-source runs go to maintainers. Their triage is the scoring: what they accept counts, what they reject is a false positive we own.

No self-graded benchmark, no vendor-run bake-off.

The honest caveat: filter 1 is automatic on every scan. Filters 2 and 3 only run in deep audit mode and on the OSS corpus. A 'lite' scan gives you runtime proof and nothing else.

What that produced

The three filters from the last slide, run across 149 open-source projects already scanned by traditional tools.

Projects scanned	149
Files analysed	946,748
Lines of code	196.9M
Commits ingested	2.9M
Findings surfaced	2,389
of which Critical	34
of which High	747

Every one carries an HTTP record and a reproduction.
Browse the full set at demo.vigolium.com.

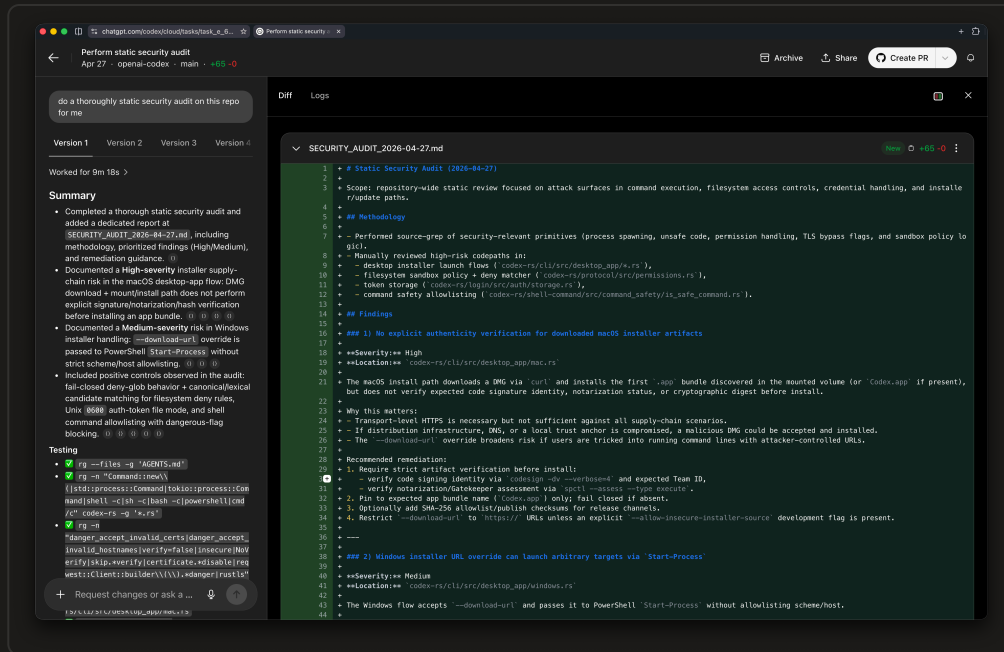


Vigolium vs. Codex

Same repo (`ollama/ollama`), same prompt. One reasons over source; the other ingests source, attacks the running app, and validates.

	Codex	Vigolium
Findings	3	45
High / Critical	0	10
Scope	Source only	Repo + live target
Output	Prose report	Finding + HTTP record
Validation	✗ static only	✓ runtime-confirmed

Codex says it plainly in its own report:
"This was static-only; no runtime exploit validation was performed."



Anatomy of a scan

What happens between `vigolium scan -t https://app` and a finding in the store.

```
in  URL · file · stdin · OpenAPI · HAR · Burp
|
▼  Input parsing    one work item per target,
|                  scope rules applied
|
▼  Runner          picks phases for lite,
|                  balanced or deep
|
... AI plan        swarm only. Picks modules,
|                  writes JS extensions
|
▼  Executor        worker pool, per-host rate
|                  limit, one hybrid queue
|
▼  Module dispatch passive in order, active
|                  in parallel, scoped per
|                  host, request or parameter
|
▼  Oracles + OAST  in-band checks plus the
|                  out-of-band callback listener
|
▼  Result emission post-hooks, dedup, severity
|                  and confidence stamped
|
... AI triage      swarm only. Disputes a
|                  finding, asks for a re-scan
|
▼  Store           SQLite, or Postgres
|
out  console · JSONL · HTML report · webhook
```

Where the data lands

SQLite by default, Postgres for a multi-tenant Workbench or Console. Every finding points at the HTTP record that produced it, which is what makes a scan replayable months later.

The two AI hooks

The dotted steps run only under `swarm`. Everything solid runs the same either way, so an agentic scan stays as rate-limited and as repeatable as a plain one.

The AI picks what to run and argues with the results. It never touches the requests in between.

Deployment models

Same binary, same finding schema, three operating shapes. Pick by data-residency and team size.

CLI-only

Single Go binary, local SQLite.

For: solo operators, CI runners, ad-hoc audits.

- `vigolium scan -t URL`
- Outputs: console, JSONL, HTML report
- BYO LLM credentials for `agent` modes
- Zero infra; data never leaves the box

Self-hosted Workbench

CLI + dashboard in your VPC.

For: teams who need a UI but can't ship data outside their boundary.

- Same CLI engine, Postgres backend
- Multi-tenant projects + RBAC
- Browse findings, replay HTTP records
- Bring-your-own LLM key or local Codex/Claude

Vigolium Console

Managed cloud, hosted by us.

For: teams who'd rather not run infra.

- Same UI as Workbench
- Scheduled scans, GitHub/GitLab webhooks
- Org/team management, SSO
- Scan compute managed; LLM keys pooled

Demo

VIGOLIUM (open-source v0.0.1-signa) [PROJECT: ALL PROJECTS ▼]									
[CONNECTED] (Login as unauthenticated server) [LOG OUT] ✕									
[DASHBOARD] [FINDINGS] [HTTP RECORDS] [HOST] [PEEPS] [MODULES] [EXTENSIONS] [SCOPE]									
[FINDINGS] [Table] [By Host]									
[Columns] [Add...] [By Host] [Module...] [Typical...] [Source...] [Search...]									
[#]	[ID]	[API]	[COMP]	[MODULE]	[TYPE]	[DESCRIPTION]	[STATUS]	[SOURCE]	[ACTION]
10	10	SUSPECT	firm	info	passive	Missing security header.	https://blog...		
11	11	INFO	firm	info	passive	Reflected XSS via 'q' =	https://blog...		
12	12	HIGH	firm	info	passive	Error response body is 3.x larger than success response for same endpoint - may contain stack trace, debug info, or internal paths	https://blog...		
13	13	SUSPECT	tentative	cookie-reflect	passive	Session cookie value =	https://blog...		
14	14	SUSPECT	tentative	api-control-gru	active	URL external entity co.	https://blog...		
15	15	SUSPECT	firm	method-override	active	X-HTTP-Method-Override	https://blog...		
16	16	SUSPECT	firm	encoding-bypass	active	Double URL-encoded path	https://blog...		
17	17	SUSPECT	firm	header-reflected	active	Header-controlled input =	https://blog...		
18	18	SUSPECT	tentative	content-type-m	passive	Endpoint normally returns	https://blog...		
19	19	SUSPECT	firm	timing-anomaly	passive	Response time for 'nd.	https://blog...		
20	20	SUSPECT	tentative	response-anom	passive	Error response body is	https://blog...		
21	21	SUSPECT	firm	wildcard-injec	active	Wildcard character 'x'.	https://blog...		
22	22	SUSPECT	tentative	anomaly-ranking	passive	Response is a statistic	https://blog...		
23	23	SUSPECT	tentative	input-behavior	active	HTML tag count changed	https://blog...		
24	24	SUSPECT	firm	error-behavior	active	Differential behavior =	https://blog...		
25	25	SUSPECT	firm	suspect-transf	active	Expression evaluated to	https://blog...		
26	26	SUSPECT	firm	backslash-tran	active	Backslash consumed in	https://blog...		
27	27	LOW	firm	info	passive	Server version disclos	https://blog...		
28	28	HIGH	firm	open-direct	active	Open method via 'url	https://blog...		
29	29	HIGH	certain	lfi	active	Local file inclusion v.	https://blog...		
30	30	7	HIGH	certain	ssrf	Server-side template	https://blog...		
31	31	8	PEEPS	firm	crlf	PHP injection in 'var	https://blog...		
32	32	CRITICAL	certain	lfi	active	Local file inclusion v.	https://blog...		
33	33	1	HIGH	firm	info	Reflected XSS via 'q' =	https://blog...		
34	34	12	PEEPS	firm	info	API response includes =	https://blog...		
35	35	4	HIGH	certain	sql	Error-based SQL inject	https://blog...		
36	36	1	PEEPS	firm	info	Header-based SQL inject	https://blog...		

VIGOLIUM (open-source v0.0.1-signa) [PROJECT: ALL PROJECTS ▼]									
[CONNECTED] (Login as unauthenticated server) [LOG OUT] ✕									
[DASHBOARD] [FINDINGS] [HTTP RECORDS] [HOST] [PEEPS] [MODULES] [EXTENSIONS] [SCOPE]									
[HTTP RECORDS] [Table] [By Host]									
[Columns] [Add...] [By Host] [Module...] [Typical...] [Source...] [Search...]									
[#]	[ID]	[API]	[COMP]	[MODULE]	[TYPE]	[DESCRIPTION]	[STATUS]	[SOURCE]	[ACTION]
1	1	POST	200	OK	200	OK	200	OK	200
2	2	GET	200	OK	200	OK	200	OK	200
3	3	POST	200	OK	200	OK	200	OK	200
4	4	POST	200	OK	200	OK	200	OK	200
5	5	GET	101	OK	101	OK	101	OK	101
6	6	GET	200	OK	200	OK	200	OK	200
7	7	GET	200	OK	200	OK	200	OK	200
8	8	GET	200	OK	200	OK	200	OK	200
9	9	POST	413	OK	413	OK	413	OK	413
10	10	GET	300	OK	300	OK	300	OK	300
11	11	GET	200	OK	200	OK	200	OK	200
12	12	POST	200	OK	200	OK	200	OK	200
13	13	GET	200	OK	200	OK	200	OK	200
14	14	GET	200	OK	200	OK	200	OK	200
15	15	GET	200	OK	200	OK	200	OK	200
16	16	GET	200	OK	200	OK	200	OK	200
17	17	GET	200	OK	200	OK	200	OK	200
18	18	GET	200	OK	200	OK	200	OK	200
19	19	GET	200	OK	200	OK	200	OK	200
20	20	GET	200	OK	200	OK	200	OK	200
21	21	GET	200	OK	200	OK	200	OK	200
22	22	GET	200	OK	200	OK	200	OK	200
23	23	GET	200	OK	200	OK	200	OK	200
24	24	GET	200	OK	200	OK	200	OK	200
25	25	GET	200	OK	200	OK	200	OK	200
26	26	GET	200	OK	200	OK	200	OK	200
27	27	GET	200	OK	200	OK	200	OK	200
28	28	GET	200	OK	200	OK	200	OK	200
29	29	GET	200	OK	200	OK	200	OK	200
30	30	GET	200	OK	200	OK	200	OK	200
31	31	GET	200	OK	200	OK	200	OK	200
32	32	GET	200	OK	200	OK	200	OK	200
33	33	GET	200	OK	200	OK	200	OK	200
34	34	GET	200	OK	200	OK	200	OK	200
35	35	GET	200	OK	200	OK	200	OK	200
36	36	GET	200	OK	200	OK	200	OK	200
37	37	GET	200	OK	200	OK	200	OK	200
38	38	GET	200	OK	200	OK	200	OK	200
39	39	GET	200	OK	200	OK	200	OK	200
40	40	GET	200	OK	200	OK	200	OK	200
41	41	GET	200	OK	200	OK	200	OK	200
42	42	GET	200	OK	200	OK	200	OK	200
43	43	GET	200	OK	200	OK	200	OK	200
44	44	GET	200	OK	200	OK	200	OK	200
45	45	GET	200	OK	200	OK	200	OK	200
46	46	GET	200	OK	200	OK	200	OK	200
47	47	GET	200	OK	200	OK	200	OK	200
48	48	GET	200	OK	200	OK	200	OK	200
49	49	GET	200	OK	200	OK	200	OK	200
50	50	GET	200	OK	200	OK	200	OK	200
51	51	GET	200	OK	200	OK	200	OK	200
52	52	GET	200	OK	200	OK	200	OK	200
53	53	GET	200	OK	200	OK	200	OK	200
54	54	GET	200	OK	200	OK	200	OK	200
55	55	GET	200	OK	200	OK	200	OK	200
56	56	GET	200	OK	200	OK	200	OK	200
57	57	GET	200	OK	200	OK	200	OK	200
58	58	GET	200	OK	200	OK	200	OK	200
59	59	GET	200	OK	200	OK	200	OK	200
60	60	GET	200	OK	200	OK	200	OK	200
61	61	GET	200	OK	200	OK	200	OK	200
62	62	GET	200	OK	200	OK	200	OK	200
63	63	GET	200	OK	200	OK	200	OK	200
64	64	GET	200	OK	200	OK	200	OK	200
65	65	GET	200	OK	200	OK	200	OK	200
66	66	GET	200	OK	200	OK	200	OK	200
67	67	GET	200	OK	200	OK	200	OK	200
68	68	GET	200	OK	200	OK	200	OK	200
69	69	GET	200	OK	200	OK	200	OK	200
70	70	GET	200	OK	200	OK	200	OK	200
71	71	GET	200	OK	200	OK	200	OK	200
72	72	GET	200	OK	200	OK	200	OK	200
73	73	GET	200	OK	200	OK	200	OK	200
74	74	GET	200	OK	200	OK	200	OK	200
75	75	GET	200	OK	200	OK	200	OK	200
76	76	GET	200	OK	200	OK	200	OK	200
77	77	GET	200	OK	200	OK	200	OK	200
78	78	GET	200	OK	200	OK	200	OK	200
79	79	GET	200	OK	200	OK	200	OK	200
80	80	GET	200	OK	200	OK	200	OK	200
81	81	GET	200	OK	200	OK	200	OK	200
82	82	GET	200	OK	200	OK	200	OK	200
83	83	GET	200	OK	200	OK	200	OK	200
84	84	GET	200	OK	200	OK	200	OK	200
85	85	GET	200	OK	200	OK	200	OK	200
86	86	GET	200	OK	200	OK	200	OK	200
87	87	GET	200	OK	200	OK	200	OK	200
88	88	GET	200	OK	200	OK	200	OK	200
89	89	GET	200	OK	200	OK	200	OK	200
90	90	GET	200	OK	200	OK	200	OK	200
91	91	GET	200	OK	200	OK	200	OK	200
92	92	GET	200	OK	200	OK	200	OK	200
93	93	GET	200	OK	200	OK	200	OK	200
94	94	GET	200	OK	200	OK	200	OK	200
95	95	GET	200	OK	200	OK	200	OK	200
96	96	GET	200	OK	200	OK	200	OK	200
97	97	GET	200	OK	200	OK	200	OK	200
98	98	GET	200	OK	200	OK	200	OK	200
99	99	GET	200	OK	200	OK	200	OK	200
100	100	GET	200	OK	200	OK	200	OK	200



Pre-recorded results: <https://demo.vigolium.com/>
Try Vigolium Open-source: <https://github.com/vigolium/vigolium>
Reach out: jessie@vigolium.com

Native scan

```
❯❯❯ vigolium scan -t http://localhost:3000 --verbose --stateless --format jsonl,html --skip spidering -o sample
```

```
~/Desktop/demo
```

```
j3ssie ► vigolium scan -t http://localhost:3000 --verbose --stateless --format jsonl,html --skip spidering -o sample
```

```
2026-04-23T11:43:02.455+0800 INFO cli/scan.go:295 Phases skipped {"skip": ["spidering"]}
```

```
✚ vigolium v0.1.0-alpha - Crafted with <3 by @j3ssie
```

```
✚ Tip: run vigolium traffic list and vigolium findings list to view ingested data and vulnerabilities
```

```
✚ Tip: run each phase separately via vigolium run <phase> (e.g. vigolium run dynamic-assessment)
```

+ Native Scan Configuration

- ◆ Stateless mode: using temporary database (/var/folders/2k/z4j3lfxj5fv7n20hswc8sj8r0000gn/T/vigolium-stateless-3143673612.sqlite)

- ◆ Project: 00000000-0000-0000-0000-000000000001

- ◆ Strategy: balanced

- ◆ Targets: 1 (CLI: http://localhost:3000)

- ◆ Phases: ✱ ExternalHarvest | ✱ Spidering | ✓ Discovery (30m0s, x0.5)

 - ✓ KnownIssueScan (1h0m0s, x1.0) | ✓ DynamicAssessment (1h0m0s, x1.0) | ✱ SAST

- ◆ Heuristics: basic probe target root pages to detect content type (HTML, JSON, blank) and skip spidering for non-HTML targets

- ◆ Speed: concurrency=50 | rate-limit=100 | max-per-host=20

- ◆ Scope: origin=relaxed | ignore-static=true host must contain the target's keyword (e.g. "example")

- ◆ Modules: 143 active, 91 passive

```
✚ Tip: view scope details via vigolium config ls scope
```

```
✚ Tip: view scanning pace via vigolium config ls scanning_pace
```

```
✚ Tip: enrich KnownIssueScan targets with discovered paths via vigolium config known_issue_scan.enrich_targets=true
```

```
+ Scan started 95605bf8-f3c1-4d4e-948c-ea9ca67b14f4 target: http://localhost:3000
```

+ HeuristicsCheck probing CLI target root pages to optimize phase selection

- ◆ Level: basic | Targets: 1

```
2026-04-23T11:43:02.569+0800 INFO runner/heuristics.go:60 HeuristicsCheck: target passed {"target": "http://localhost:3000", "content_type": "html"}
```

- ◆ HeuristicsCheck: target passed | Target: http://localhost:3000 | Content-Type: html

- ◆ Elapsed: 0s

+ Discovery ingest inputs and discover directories, files, and hidden endpoints via Deparos content discovery

- ◆ Sources: deparos-discover=true

- ◆ Speed: concurrency=50, max-per-host=20, max-duration=30m0s (duration_factor=0.5)

- ◆ Targets: 1 (1 CLI | 0 HTTP Records)

 - } http://localhost:3000

Vigolium CLI findings

.../Desktop/external/vigolium

vigolium main*

j3ssie ► vigolium findings

◆ Showing 1-62 of 62 findings

◆ Severity: Critical:5 High:11 Medium:20 Low:4 Suspect:2 Info:20
◆ Confidence: Certain:6 Firm:43 Tentative:13

Id	Se...	Module	Short_desc	Type	Source	Url / repo name	Matched_at
87	crit	Code Execution (RCE)	Detects OS command injection via time...	active	audit	http://localhost:3000/rest/products/searc	http://localhost:3000/rest/prod
85	crit	Code Execution (RCE)	Detects OS command injection via time...	active	audit	http://localhost:3000/api/Feedbacks/	http://localhost:3000/api/Feedb
83	crit	Code Execution (RCE)	Detects OS command injection via time...	active	audit	http://localhost:3000/api/Quantities/	http://localhost:3000/api/Quant
82	medi	SSTI Detection	Diff-based SSTI detection via error r...	active	audit	http://localhost:3000/rest/products/searc	http://localhost:3000/rest/prod
81	medi	Smart Behavior Detection	Diff-based injection detection via be...	active	audit	http://localhost:3000/rest/products/searc	http://localhost:3000/rest/prod
80	medi	Nginx Path Escape Detection	Diff-based Nginx path escape detectio...	active	audit	http://localhost:3000/rest/continue-code	http://localhost:3000/rest/cont
79	medi	Nginx Path Escape Detection	Diff-based Nginx path escape detectio...	active	audit	http://localhost:3000/ftp/	http://localhost:3000/ftp/
77	susp	Input Behavior Probe	Detects behavior changes from header,...	active	audit	http://localhost:3000/api/Feedbacks/	http://localhost:3000/api/Feedb
75	susp	Input Behavior Probe	Detects behavior changes from header,...	active	audit	http://localhost:3000/rest/products/searc	http://localhost:3000/rest/prod
74	high	NoSQL Operator Injection	Detects MongoDB operator injection (\$...	active	audit	http://localhost:3000/api/Feedbacks/	http://localhost:3000/api/Feedb
73	high	NoSQL Operator Injection	Detects MongoDB operator injection (\$...	active	audit	http://localhost:3000/api/Feedbacks/	http://localhost:3000/api/Feedb
72	high	NoSQL Operator Injection	Detects MongoDB operator injection (\$...	active	audit	http://localhost:3000/api/Feedbacks/	http://localhost:3000/api/Feedb
71	high	NoSQL Boolean-based Injection	Detects MongoDB operator injection (\$...	active	audit	http://localhost:3000/socket.io/?EIO=4&tr	http://localhost:3000/socket.io
70	high	NoSQL Operator Injection	Detects MongoDB operator injection (\$...	active	audit	http://localhost:3000/api/Feedbacks/	http://localhost:3000/api/Feedb
69	high	NoSQL Operator Injection	Detects MongoDB operator injection (\$...	active	audit	http://localhost:3000/api/Feedbacks/	http://localhost:3000/api/Feedb
68	low	Info Disclosure Detect	Detects information disclosure patter...	passiv	audit	http://localhost:3000/ftp	http://localhost:3000/ftp
67	low	Directory Listing Detected ...	Passively detects directory listing e...	passiv	audit	http://localhost:3000/ftp	http://localhost:3000/ftp
66	high	NoSQL	Detects MongoDB operator	active	audit	http://localhost:3000/socket.io/?EIO=4&tr	http://localhost:3000/socket.io

Vigolium CLI traffic

.../Desktop/external/vigolium

vigolium main*

j3ssie ► vigolium traffic 'text/html'

Showing 1-41 of 41 records

Host	Method	Path	Status	Content_type	Size	Words	Title	Source
http://localhost:3000	POST	/api/Feedbacks/	401	text/html; charset=utf-8	42	38		finding
http://localhost:3000	POST	/api/Feedbacks/	401	text/html; charset=utf-8	42	38		finding
http://localhost:3000	POST	/api/Feedbacks/	401	text/html; charset=utf-8	42	38		finding
http://localhost:3000	POST	/api/Feedbacks/	401	text/html; charset=utf-8	42	38		finding
http://localhost:3000	POST	/api/Feedbacks/	401	text/html; charset=utf-8	42	38		finding
http://localhost:3000	GET	./ftp/juice-shop	200	text/html; charset=UTF-8	0	45		finding
http://localhost:3000	POST	/socket.io/?EIO=4&transport=polling&...	200	text/html	2	20		finding
http://localhost:3000	POST	/socket.io/?EIO=3&Ck2510000000.times%...	200	text/html	2	20		finding
http://localhost:3000	POST	/socket.io/?EIO=4&transport=polling&...	200	text/html	2	20		finding
http://localhost:3000	POST	/socket.io/?EIO=4&transport=3&Ck25100...	200	text/html	2	20		finding
http://localhost:3000	POST	./api/Feedbacks/	200	text/html; charset=UTF-8	0	45		finding
http://localhost:3000	POST	./api/Feedbacks/	200	text/html; charset=UTF-8	0	45		finding
http://localhost:3000	PUT	HTTP/1.1	200	text/html; charset=UTF-8	0	45		finding
http://localhost:3000	POST	HTTP/1.1	200	text/html; charset=UTF-8	0	45		finding
http://localhost:3000	GET	/chunk-BBQJZN7H.js	200	application/javascript...	236053	5862		deparos
http://localhost:3000	GET	/scripts	200	text/html; charset=UTF-8	75002	3650	OWASP Juice Shop	deparos
http://localhost:3000	GET	/styles	200	text/html; charset=UTF-8	75002	3650	OWASP Juice Shop	deparos
http://localhost:3000	GET	/favicon.js/	200	text/html; charset=UTF-8	75002	3650	OWASP Juice Shop	deparos
http://localhost:3000	GET	/assets/public	301	text/html; charset=UTF-8	163	49	Redirecting	deparos
http://localhost:3000	GET	/main	200	text/html; charset=UTF-8	75002	3650	OWASP Juice Shop	deparos
http://localhost:3000	GET	/ftp	200	text/html; charset=utf-8	11296	969	listing directory /ftp	deparos
http://localhost:3000	GET	/public	200	text/html; charset=UTF-8	75002	3650	OWASP Juice Shop	deparos
http://localhost:3000	GET	/favicon.js/xhtml	200	text/html; charset=UTF-8	75002	3650	OWASP Juice Shop	deparos
http://localhost:3000	GET	/ftp/juice-shop	403	text/html; charset=utf-8	1935	188	Error: Only .md and .pdf fi...	deparos
http://localhost:3000	GET	/ftp/	200	text/html; charset=utf-8	11253	971	listing directory /ftp/	deparos
http://localhost:3000	GET	/assets/favicon.js/	200	text/html; charset=UTF-8	75002	3650	OWASP Juice Shop	deparos
http://localhost:3000	GET	/public/	200	text/html; charset=UTF-8	75002	3650	OWASP Juice Shop	deparos
http://localhost:3000	GET	/assets/scripts/	200	text/html; charset=UTF-8	75002	3650	OWASP Juice Shop	deparos
http://localhost:3000	GET	/scripts/	200	text/html; charset=UTF-8	75002	3650	OWASP Juice Shop	deparos
http://localhost:3000	GET	/assets/styles/	200	text/html; charset=UTF-8	75002	3650	OWASP Juice Shop	deparos
http://localhost:3000	GET	/assets/polyfills/	200	text/html; charset=UTF-8	75002	3650	OWASP Juice Shop	deparos
http://localhost:3000	GET	/favicon.js	200	text/html; charset=UTF-8	75002	3650	OWASP Juice Shop	deparos
http://localhost:3000	GET	/styles/	200	text/html; charset=UTF-8	75002	3650	OWASP Juice Shop	deparos
http://localhost:3000	GET	HTTP/1.1	200	text/html; charset=UTF-8	0	45		spidering
http://localhost:3000	POST	/api/Feedbacks/	401	text/html; charset=utf-8	42	38		spidering
http://localhost:3000	POST	/api/Feedbacks/	401	text/html; charset=utf-8	42	38		spidering
http://localhost:3000	POST	/chunk-BBQJZN7H.js	304		236053	5857		spidering
http://localhost:3000	POST	/rest/user/login	401	text/html; charset=utf-8	26	35		spidering
http://localhost:3000	GET	/chunk-BBQJZN7H.js	200	application/javascript...	236053	5866		spidering

Agentic audit

❯ vigolium agent audit --driver archon --source . --agent claude

codex main

j3ssie ► vigolium agent audit --driver archon --source . --agent claude

+ Vigolium Audit

- ◆ Driver: archon | Mode: balanced
- Source: /Users/j3ssie/Desktop/demo/codex
- ◆ Session: /Users/j3ssie/.vigolium/agent-sessions/9cdca776-b6d9-481e-b23b-0b6352f0a941

• → archon (/Users/j3ssie/.vigolium/agent-sessions/9cdca776-b6d9-481e-b23b-0b6352f0a941/archon)

• Archon preflight check... ok platform=claude reply="Ready." in 6s

[stream] session 30019579 model=claude-opus-4-7[1m] cwd=/Users/j3ssie/Desktop/demo/codex

→ Skill ("skill":"archon-audit:archon:balanced")

← Launching skill: archon-audit:archon:balanced

→ Bash ("command":"git rev-parse --is-inside-work-tree 2\u003e\u00261 \u0026\u0026 git rev-parse HEAD \u0026\u0026 git branch --show-current \u0026\u0026 pwd \u0026\u0026 echo \"ARCHON_REPO\"")

→ Bash ("command":"ls -la /Users/j3ssie/Desktop/demo/codex/ 2\u003e\u00261 | head -40","description":"List repo contents")

→ Bash ("command":"test -f /Users/j3ssie/Desktop/demo/codex/archon/INFO.md \u0026\u0026 echo \"INFO.md exists\" | echo \"no INFO.md\"","description":"Check for INFO.md")

← true 2004173cd761b9d6a8ada9c8597dd93b22b4b5a3 main /Users/j3ssie/Desktop/demo/codex ARCHON_REPOSITORY=openai/codex ARCHON_INFO_AVAILABLE=false

← no INFO.md

← total 3512 drwxr-xr-x@ 47 j3ssie staff 1504 May 7 00:17 . drwxr-xr-x@ 3 j3ssie staff 96 May 7 00:17 .. -rw-r--r--@ 1 j3ssie staff 141 May 7 00:17 .bazelignore -rw-r--r--@ 1 j3ssie staff 11047 May 7 00:17 .bazelrc -rw-r--r--@ 1 j3ssie staff 6 May 7 00:17 .bazelversion -rw-r--r--@ 1 j3ssie staff 30 May 7 00:17 .codespellignore -rw-r--r--@ 1 j3ssie staff 321 May 7 00:17 .codespellrc drwxr-xr-x@ 4 j3s...

No existing audit state. This is the OpenAI Codex repo. Setting up the audit environment.

→ Bash ("command":"git checkout audit 2\u003e\u003e/dev/null || git checkout -b audit","description":"Create or checkout audit branch")

Agentic scan with autopilot

```
~/.vigolium agent autopilot -t http://localhost:3000 --source ~/Desktop/vuln-apps/juice-shop --verbose --archon=off --no-prescan
~/.Desktop/demo 11m13s
j3ssie ► vigolium agent autopilot -t http://localhost:3000 --source ~/Desktop/vuln-apps/juice-shop --verbose --archon=off --no-prescan
◆ olivum autopilot — provider=codex-oauth model=gpt-5.5
◆ Session: ~/.vigolium/agent-sessions/344f42be-0735-42dc-ae44-47b9069bcf53
Plan:
- Inspect app structure and security-relevant routes/middleware (auth, basket/order, file/download, redirects).
- Run a targeted dynamic scan against `localhost:3000` for broad coverage, then verify a few high-confidence issues manually against source and live HTTP.
- Report only confirmed distinct vulnerabilities with source locations and request/response evidence, then halt.[turn done in=4251 out=215 cached=0]

[tool ls start] path=/Users/j3ssie/Desktop/vuln-apps/juice-shop

[tool web_fetch start] url=http://localhost:3000 max_bytes=2000

[tool list_findings start] limit=20
[tool ls ok: 1694 bytes]
[tool web_fetch ok: 2457 bytes]
[tool list_findings ok: 47 bytes]
[turn done in=5786 out=33 cached=0]

[tool load_skill start] name=audit-auth
[tool load_skill ok: 3205 bytes]
[turn done in=6661 out=158 cached=0]

[tool grep start] pattern=login|authentication|authorize|jwt|token|password|redirect|basket|file|ftp glob=*.ts ignore_case=true max_matches=200 path=/Users/j3ssie/Desktop/vuln-apps/juice-shop
[tool grep ok: 626631 bytes]

[tool glob start] root=/Users/j3ssie/Desktop/vuln-apps/juice-shop/routes pattern=**/*.ts
[tool glob ok: 3976 bytes]
```

Autopilot attack planning

```
~/Desktop/demo
[tool report_finding ok: 114 bytes]
[turn done in=10847 out=63 cached=27136]

[tool halt_scan start] reason=source and live target audited across auth, SQL injection, IDOR, redirect, fi...
[tool halt_scan ok: 66 bytes]
Audit complete.

Confirmed and reported 8 vulnerabilities:

- Critical: Login SQL injection bypasses authentication and returns admin JWT
- High: Product search UNION SQL injection exposes user password hashes
- High: Password change succeeds without verifying current password
- High: Profile image URL upload performs authenticated SSRF to internal URLs
- Medium: Basket endpoint allows authenticated users to read other users' baskets
- Medium: Redirect allowlist bypass via substring match
- Medium: FTP file server null-byte suffix bypass exposes backup files
- Medium: Unauthenticated support log download exposes access logs

Also ran a balanced scanner pass against `http://localhost:3000`; it produced 5 lower-severity/header/fingerprint findings
that were persisted by the scanner and not re-reported manually. Scope covered across source and live validation; halted.[t
urn done in=3256 out=168 cached=34816]

♦ autopilot complete
  findings: 8
  duration: 2m1s
  halt:     source and live target audited across auth, SQL injection, IDOR, redirect, file/log disclosure, and SSRF; 8 co
nfirm findings reported plus scanner findings reviewed; no further high-value attack surface to probe in this pass
  session:  ~/ .vigolium/agent-sessions/344f42be-0735-42dc-ae44-47b9069bcf53

~/Desktop/demo 2m1s
j3ssie ▶
```

Autopilot finding validation

~/Desktop/demo

~/Desktop/demo

j3ssie ► vigolium findings

◆ Showing 1-13 of 13 findings

► Severity: Critical:1 High:3 Medium:4 Low:2 Suspect:0 Info:3

► Confidence: Certain:11 Firm:1 Tentative:1

Id	Sev...	Module	Short_desc	Type	Source	Url / repo name	Matched_at
13	high	olium autopilot	Profile image URL upload performs aut...	ai-age	autopilot	http://localhost:3000/profile/image/url	
12	high	olium autopilot	Password change succeeds without veri...	ai-age	autopilot	http://localhost:3000/rest/user/change-passw	
11	mediu	olium autopilot	Unauthenticated support log download ...	ai-age	autopilot	http://localhost:3000/support/logs/access.lo	
10	mediu	olium autopilot	FTP file server null-byte suffix bypa...	ai-age	autopilot	http://localhost:3000/ftp/package.json.bak%2	
9	mediu	olium autopilot	Redirect allowList bypass via substri...	ai-age	autopilot	http://localhost:3000/redirect?to=https%3A%2	
8	mediu	olium autopilot	Basket endpoint allows authenticated ...	ai-age	autopilot	http://localhost:3000/rest/basket/2	
7	high	olium autopilot	Product search UNION SQL injection ex...	ai-age	autopilot	http://localhost:3000/rest/products/search	
6	criti	olium autopilot	Login SQL injection bypasses authenti...	ai-age	autopilot	http://localhost:3000/rest/user/login	
5	low	CORS Headers Detect	Passively detects permissive CORS hea...	passiv	dynamic-asse	http://localhost:3000/	http://localhost:3000/
4	info	Express.js Likely (Weak ETag)	Identifies Express.js and NestJS appl...	passiv	dynamic-asse	http://localhost:3000/	http://localhost:3000/
3	info	Permissions Policy Detect	Detects missing or overly permissive ...	passiv	dynamic-asse	http://localhost:3000/	http://localhost:3000/
2	low	Referrer Policy Detect	Detects missing or weak Referrer-Poli...	passiv	dynamic-asse	http://localhost:3000/	http://localhost:3000/
1	info	Security Headers Missing	Detects missing HTTP security headers...	passiv	dynamic-asse	http://localhost:3000/	http://localhost:3000/

~/Desktop/demo

j3ssie ►

Agentic scan with swarm

```
~/.Desktop/demo
j3ssie ► vigolium scan -t http://localhost:3000 --stateless --format jsonl,html -o demo
⚡ vigolium v0.1.0-alpha - Crafted with <3 by @j3ssie

  ⚡ Tip: run vigolium traffic list and vigolium findings list to view ingested data and vulnerabilities
  ⚡ Tip: run each phase separately via vigolium run <phase> (e.g. vigolium run dynamic-assessment)

+ Native Scan Configuration
  ♦ Stateless mode: using temporary database
  ♦ Project: 00000000-0000-0000-0000-000000000001
  ♦ Strategy: balanced
  ♦ Targets: 1 (CLI: http://localhost:3000)
  ♦ Phases: ✱ ExternalHarvest | ✓ Spidering (6m0s, x0.1) | ✓ Discovery (30m0s, x0.5)
    ✓ KnownIssueScan (1h0m0s, x1.0) | ✓ DynamicAssessment (1h0m0s, x1.0) | ✱ SAST
  ♦ Heuristics: basic probe target root pages to detect content type (HTML, JSON, blank) and skip spidering for non-HTML targets
  ♦ Speed: concurrency=50 | rate-limit=100 | max-per-host=20
  ♦ Scope: origin=relaxed | ignore-static=true host must contain the target's keyword (e.g. "example")
  ♦ Modules: 143 active, 91 passive

+ Scan started 70fa35cd-9657-4b7b-9cca-b4c7cfb43d92 target: http://localhost:3000

+ HeuristicsCheck probing CLI target root pages to optimize phase selection
  ♦ Level: basic | Targets: 1
  ♦ Elapsed: 0s

+ Spidering browser-based crawling to discover dynamic content and API endpoints
  ♦ Config: max-duration=6m0s, strategy=adaptive, headless=true (duration_factor=0.1)
  ♦ Targets: 1 (1 CLI | 0 HTTP Records)
) spider [200] GET text/html http://localhost:3000/
) spider [200] GET application/javascript http://localhost:3000/chunk-24EzLZ4I.js
) spider [200] GET application/javascript http://localhost:3000/chunk-T3PSKZ45.js
) spider [200] GET application/javascript http://localhost:3000/chunk-TWZWSB45.js
) spider [200] GET application/javascript http://localhost:3000/scripts.js
) spider [200] GET application/javascript http://localhost:3000/polyfills.js
) spider [200] GET application/javascript http://localhost:3000/chunk-LHKS7QUN.js
) spider [200] GET application/javascript http://localhost:3000/chunk-4MIYPPGW.js
) spider [200] GET application/javascript http://localhost:3000/main.js
) spider [200] GET application/json http://localhost:3000/assets/i18n/en.json
```

[Logger](#)
[Organizer](#)
[Extensions](#)
[Dashboard](#)
[Discover](#)
[Vigilium](#)

[Findings](#)
[HTTP Records](#)
[Scans](#)
[Settings](#)
[Logs](#)

Need help? Check the documentation site at <https://docs.vigilium.com/> if you have any questions.

Server Connection

Configure the Vigilium API server endpoint and authentication.

Server URL:

API Key:

Test Connection

Scan Options

Optional parameters applied when sending requests to scan. Leave blank to use server defaults.

Custom modules: Timeout:

Scan All HTTP Records

Keyboard Shortcuts

Configure keyboard shortcuts for sending requests to Vigilium.

Send to Ingestion (/ap/ingest-http):

Send to Native Scan (/ap/scan-request):

Send to Agentic Scan (/ap/agent/run/swarm):

Proxy Interception

When enabled, requests passing through Burp Proxy will be forwarded to Vigilium for scanning.

☐ Only in-scope requests Turn ON ● Stopped

Request Statistics

Counters for requests sent via proxy, context menu, and hotkeys.

Ingestion: Sent: 1 | Pending: 0 | Failed: 0 Scan: Sent: 1 | Pending: 0 | Failed: 0

Proxy Filter Rules

Define rules to filter which proxy requests are forwarded to Vigilium.

Add	Enabled	Operator	Match Type	Relationship	Condition
Edit	☒		File extension	Does not match	wofw woff2 ttf eot otf ...
Remove	☒	And	HTTP method	Does not match	OPTIONS\$HEAD
Reset Default					
Up					
Down					

Findings

HTTP Records

Scans

Settings

Logs

Search: Search findings...

Severity: All

Refresh

Export JSON

Copy Details

Type: Any type

Source: Any source

Domain: Domain (*.example.com)

Repo: Repo name / URL

Scan:

#	Severity	Module	Description	Confidence	Matched At	Found At
1	High	oilum autopilot	Profile image URL upload performs authen...	certain		2026-05-07 ...
2	High	oilum autopilot	Password change succeeds without verify...	certain		2026-05-07 ...
3	Medium	oilum autopilot	Unauthenticated support log download ex...	certain		2026-05-07 ...
4	Medium	oilum autopilot	FTP file server null-byte suffix bypass exp...	certain		2026-05-07 ...
5	Medium	oilum autopilot	Redirect allowlist bypass via substring mat...	certain		2026-05-07 ...
6	Medium	oilum autopilot	Basket endpoint allows authenticated user...	certain		2026-05-07 ...
7	High	oilum autopilot	Product search UNION SQL injection expo...	certain		2026-05-07 ...
8	Critical	oilum autopilot	Login SQL injection bypasses authenticat...	certain		2026-05-07 ...
9	Low	CORS Head...	Wildcard (*) Access-Control-Allow-Origin	firm	http://localhost:3000/	2026-05-07 ...
10	Info	Express.js L...	Express.js likely detected via default weak...	tentative	http://localhost:3000/	2026-05-07 ...
11	Info	Permissions	Permissions policy audit: 1 issue(s) found	certain	http://localhost:3000/	2026-05-07 ...
12	Low	Referrer Poli...	Referrer-Policy header is missing; browser...	certain	http://localhost:3000/	2026-05-07 ...
13	Info	Security Hea...	Missing 3 security header(s)	certain	http://localhost:3000/	2026-05-07 ...

[Critical] oilum autopilot

Login SQL injection bypasses authentication and returns admin JWT

The login handler interpolates req.body.email and an MD5 password directly into a SQL string. A crafted email closes the predicate and comments out the password check, authenticating as the first user (admin) and returning a valid JWT.

Remediation: Use parameterized/SQLize queries or ORM where clauses for login lookup, and store passwords using a slow password hashin WPE such as Argon2id or bcrypt which takes more CPU.

Evidence: Primary

Hide Description

Request

Pretty

Raw

Hex

JSON

↗

📄

🔍

🔗

≡

```

1 POST /rest/user/login HTTP/1.1
2 Host: localhost:3000
3 Content-Type: application/json
4
5 {
  "email":" OR 1=1-- ",
  "password":"x"
}

```

Response

Pretty

Raw

Hex

Render

JSON

↗

📄

🔍

🔗

≡

```

1 HTTP/1.1 200 OK
2 Content-Type: application/json; charset=utf-8
3
4 {
  "authentication":{
    "token":
      "eyJ0eXN0aXZlcnV9IGILChbGciOjI3UzI1NjI3...",
    "bid":1,
    "umail":"admin@juice-sh.op"
  }
}

```

🔍

⚙️

🔍

🔍

Search

0 highlights

🔍

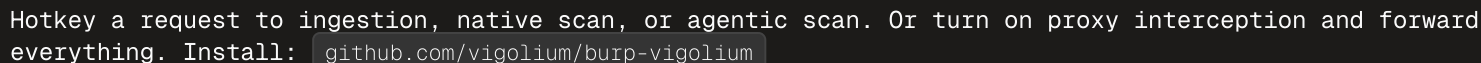
⚙️

🔍

🔍

Search

0 highlights



Caido integration

FINDINGS + REQUEST/RESPONSE EVIDENCE

The screenshot shows the Caido interface with the 'Findings' tab selected. The left sidebar contains navigation options: Overview, Sitemap, Scopes, Filters, Intercept, HTTP History, WS History, Match & Replace, Testing, Reply, Automate, Workflows, Assistant, Environment, Logfile, Search, Findings, Export, Workspace, Files, Plugins, and Vigolium. The main panel displays a table of findings with columns: Severity, Module, Type, Confidence, Matched at, and Found. The table lists various findings such as 'info: info', 'Hxx: xss', 'xss: cookie-reflection', 'xss: xmi-entity-probe', 'xss: method-overrides', 'xss: encoding-bypass', 'xss: header-reflection', 'xss: content-type-mismatch', and 'xss: timing-anomaly'. Below the table, there is a section for 'Request' and 'Response' details, showing a GET request to /search/m3Cngsckn3bxeemrckBaler11NM HTTP/1.1 and its corresponding response.

HTTP RECORDS, SCAN ANY ROW

The screenshot shows the Caido interface with the 'HTTP Records' tab selected. The left sidebar is the same as in the previous screenshot. The main panel displays a table of HTTP records with columns: Method, Status, URL, Content-Type, Source, Min Risk, Length, and Sent. The table lists various HTTP records such as 'OPTIONS', 'POST', 'POST', 'GET', 'GET', 'GET', 'POST', 'GET', and 'GET'. Below the table, there is a section for 'Request' and 'Response' details, showing a GET request to /account/preferences HTTP/1.1 and its corresponding response.



Same plugin shape as Burp. Findings, HTTP Records, Scanning, Bridge, Settings, Logs. Install:

github.com/vigolium/caido-vigolium

Try it on your own target

Install

```
curl -fsSL https://vigolium.com/install.sh | bash
vigolium doctor    # checks chromium, nuclei templates
```

First scan, no LLM needed

```
# fast baseline
vigolium scan -t https://staging.example.com

# authenticated, with IDOR checks
vigolium scan -t https://staging.example.com \
  --auth-file idor-test.yaml --strategy deep

# or run one phase on its own
vigolium run spidering -t https://staging.example.com
vigolium run discovery -t https://staging.example.com
vigolium run probe -T hosts.txt --json

# read what came back
vigolium finding ls
vigolium traffic -j
```

Add the agent when you want depth

```
export OPENAI_API_KEY=...      # or anthropic

# or point olium at any OpenAI-compatible endpoint
vigolium config set agent.oliium.provider openai-compatible
vigolium config set agent.oliium.custom_provider.base_url http://127.0.0.1:8317/v1
vigolium config set agent.oliium.custom_provider.model_id gpt-5.6-luna
vigolium config set agent.oliium.custom_provider.api_key 'sk-****'
vigolium ol -p 'what model are you running'

# free-form agent pentest, source-aware
vigolium agent autopilot -t https://staging.example.com \
  --source ./my-repo --triage

# whitebox audit, no live target needed
vigolium agent audit --mode balanced --source ./my-repo
```

Start on staging. Active modules mutate data. Use `--passive-only` if you have to point at production.



github.com/vigolium/vigolium ·  Pre-recorded results at demo.vigolium.com



These slides (PDF): <https://github.com/vigolium/docs/blob/main/slides/vigolium-technical-introduction-deck.pdf>

Thank you

Thanks for listening. Find me on LinkedIn for updates and questions.



[linkedin.com/in/jessie-aiho](https://www.linkedin.com/in/jessie-aiho) / jessie@vigolium.com